TCP vs QUIC

A Practical Comparison through Chat Applications

Computer Networks Lab

Undergraduate Computer Networks Course

TCP QUIC Python Java

Agenda

1. TCP — Transmission Control Protocol
2. QUIC — Quick UDP Internet Connections
3. Protocol Comparison
4. TCP Implementation
5. QUIC Implementation
6. Code Comparison
7. Key Takeaways

TCP — Transmission Control Protocol

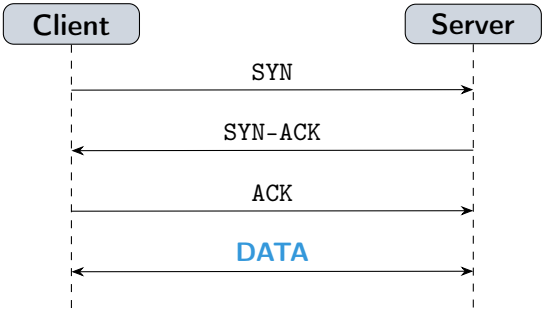
Core guarantees (RFC 793 / 9293)

- Ordered delivery
- Reliable — retransmission on loss
- Flow control (sliding window)
- Congestion control (slow start, AIMD)

Limitations

- 1 RTT for 3-way handshake
- **Head-of-line blocking** — one lost packet stalls all data
- Connection tied to IP:port (no migration)
- TLS is a separate layer (+1 RTT)

TCP — Three-Way Handshake



Connection overhead

- 3 packets exchanged before any data
- = **1 RTT overhead**

Adding TLS on top

TCP handshake	+1 RTT
TLS 1.2	+2 RTT
TLS 1.3	+1 RTT
TLS 1.3 over TCP	**2 RTT total**

QUIC fixes this: transport + TLS in **1 RTT**

QUIC — Quick UDP Internet Connections

Design goals (RFC 9000, 2021)

- Fix TCP's head-of-line blocking
- Faster connection setup (0–1 RTT)
- Encrypt everything by default
- Survive IP address changes

How it works

- Runs **over UDP** — custom reliability
- **TLS 1.3 built-in** and mandatory
- **Multiple independent streams** per connection
- **Connection ID** — survives NAT rebinding

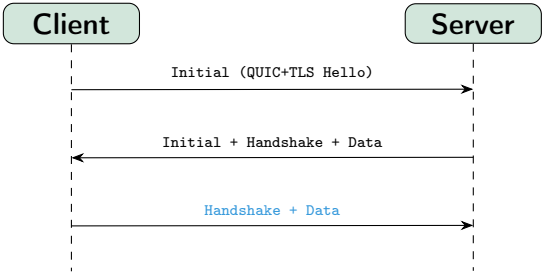
Used by

HTTP/3, YouTube, Google Search, Cloudflare, Meta, Apple iCloud

Key difference from TCP

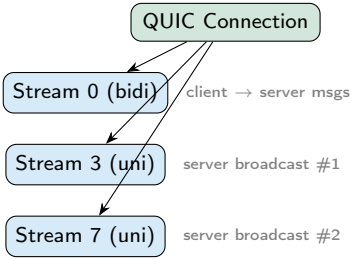
A lost packet only blocks *its own stream*, not all other streams on the connection.

QUIC — Connection Setup & Stream Model



Transport + TLS in **1 RTT**

QUIC stream types

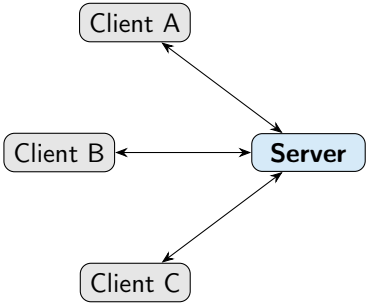


- Loss in stream 3 does **not** block stream 7
- Client-initiated or server-initiated
- Unidirectional or bidirectional

TCP vs QUIC — Protocol Comparison

Property	TCP	QUIC
Runs over	IP directly	UDP
Connection setup	1 RTT (3-way handshake)	1 RTT (combined w/ TLS)
Encryption	Optional (TLS on top)	Mandatory TLS 1.3
Streams/conn	1 ordered byte stream	Many independent streams
HoL blocking	Yes — stalls all data	No — streams independent
Conn migration	No — tied to IP:port	Yes — connection ID
Standard	RFC 793 (1981)	RFC 9000 (2021)
Used by	HTTP/1.1, HTTP/2, SSH	HTTP/3, YouTube, CDNs

Chat Application — Architecture



broadcasts to all others

Message flow

1. Client connects and sends **name**
2. Client sends a **message**
3. Server **broadcasts** to all other clients
4. Client disconnects — server notifies others

Protocol	Lang	Files
TCP	Python	server_tcp.py / client_tcp.py
TCP	Java	ServerTCP.java / ClientTCP.java
QUIC	Python	server.py / client.py
QUIC	Java	ServerQUIC.java / ClientQUIC.java

TCP Server — Python

Python

```python
# One coroutine handles one client connection
async def handle_client(reader, writer):
    conn_id = _alloc_id()
    _writers[conn_id] = writer

    while True:
        data = await reader.readline() # suspend
            until '\n'
        if not data:
            break                          # client
                disconnected

        text = data.decode().strip()
        if username is None:
            username = text                # first
                msg = display name
            _broadcast(f"*** {username} joined
                ***", conn_id)
        else:
            _broadcast(f"{username}: {text}",
                conn_id)
```

Key points

- `readline()` suspends — other clients still run
- `b""` means connection closed
- First line = display name (app protocol)
- **No threads** — coroutines share one thread

TCP Client — Python

Python

```python
# TCP 3-way handshake
reader, writer = await asyncio.open_connection(
    HOST, PORT)

# Task 1: receive broadcasts from server
async def receive_loop():
    while True:
        data = await reader.readline()
        print(data.decode().strip())

# Task 2: read keyboard and send to server
async def send_loop():
    while True:
        line = await loop.run_in_executor(None,
            stdin.readline)
        writer.write((line + "\n").encode())

# Run both concurrently; stop when either
    finishes
await asyncio.wait(
    [create_task(receive_loop()), create_task(
        send_loop())],
```

Key points

- `open_connection` does the handshake
- Two tasks run concurrently in **one thread**
- `run_in_executor` offloads blocking stdin to a thread pool
- Stop when user types `quit`

TCP Server — Java Java

```java
// Accept loop      one new thread per incoming
    connection
while (true) {
    Socket socket = serverSocket.accept(); //
        blocks
    int id = allocId();
    ClientHandler handler = new ClientHandler(
        socket, id);
    clients.put(id, handler);       //
        ConcurrentHashMap
    new Thread(handler).start();    // OS-
        scheduled thread
}

// Inside ClientHandler.run()
while ((line = reader.readLine()) != null) {
    if (username == null) {
        username = line;
        broadcast("*** " + username + " joined
            ***", connId);
    } else {
        broadcast(username + ": " + line, connId)
```

Key points

- **One thread per client** (preemptive)
- `ConcurrentHashMap` needed — multiple threads access client list
- Python uses a plain dict — safe because only 1 async thread

Python vs Java

Same protocol logic.
Different concurrency model: coroutines vs threads.

QUIC Server — Python

Python

```
class ChatServerProtocol(QuicConnectionProtocol):

    # Called for every QUIC event on this
        connection
    def quic_event_received(self, event):
        if isinstance(event, StreamDataReceived):
            self._buf += event.data
            while b"\n" in self._buf:        #
                buffer until '\n'
                line, self._buf = self._buf.split
                    (b"\n", 1)
                self._on_line(line.decode())

    # Broadcast: open a new stream per
        destination
    def _broadcast(self, message, exclude_id):
        for proto in active_connections.values():
            sid = proto._quic.
                get_next_available_stream_id(
                      is_unidirectional=True)
            proto._quic.send_stream_data(sid,
                data, end_stream=True)
```

vs TCP server

- No `readline()` — data arrives as **events**
- Must buffer bytes manually until `\n`
- Each broadcast opens a **new QUIC stream**
- One class instance = one connection

QUIC streams are cheap.
Opening one per broadcast is normal and efficient.

QUIC Client — Python

Python

```
class ChatClientProtocol(QuicConnectionProtocol):
    # Server pushes messages via new streams
        handle here
    def quic_event_received(self, event):
        if isinstance(event, StreamDataReceived):
            text = event.data.decode().strip()
            print(f"\r{text}\n> ", end="", flush=
                True)
        else:
            super().quic_event_received(event)

# Configure: skip cert check (classroom demo)
config = QuicConfiguration(is_client=True)
config.verify_mode = ssl.CERT_NONE

async with connect(HOST, PORT, configuration=
    config,
                   create_protocol=
                       ChatClientProtocol) as
                       conn:
    reader, writer = await conn.create_stream() #
        bidi stream
```

Key points

- Subclass `QuicConnectionProtocol`
- Server pushes via `StreamDataReceived`
- Outgoing: client opens a **bidirectional** stream
- `noServerCertificateCheck` skips TLS cert validation

QUIC Server — Java Java (kwik library)

```
// Factory: kwik calls createConnection() per new
    client
class ChatProtocolFactory
    implements
        ApplicationProtocolConnectionFactory {

  public ApplicationProtocolConnection
      createConnection(
        String protocol, QuicConnection conn) {
    return new ChatSession(idCounter.
        incrementAndGet(), conn);
  }
  public int
      maxConcurrentPeerInitiatedBidirectionalStrea
      () {
    return 10;
  }
}

// kwik calls this when client opens a stream
public void acceptPeerInitiatedStream(QuicStream
    stream) {
```

Key points

- **ALPN** `"chat"` is registered — QUIC uses this during TLS handshake
- `acceptPeerInitiatedStream` ≈ `quic_event_received` in Python
- kwik wraps QUIC stream in `InputStream` — feels like TCP!
- Broadcast: `createStream(false)` = unidirectional push

Code Comparison — Receiving Data

TCP **TCP — Python**

```
data = await reader.readline()
# blocks until '\n' arrives
text = data.decode().strip()
```

TCP **TCP — Java**

```
String line = reader.readLine();
// blocks until '\n' arrives
```

QUIC **QUIC — Python**

```
def quic_event_received(self, event):
    if isinstance(event,
        StreamDataReceived):
        self._buf += event.data
        while b"\n" in self._buf:
            line, self._buf = \
                self._buf.split(b"\n",
                    1)
```

QUIC **QUIC — Java (kwik)**

```
// kwik wraps stream in InputStream
BufferedReader r = new BufferedReader(
  new InputStreamReader(
    stream.getInputStream()));
String line = r.readLine(); // like TCP!
```

TCP gives a continuous byte stream. QUIC delivers data as **events** — you must buffer. kwik's Java API hides this with `InputStream`.

Code Comparison — Broadcasting to Other Clients

TCP **TCP — Python**

```python
for cid, writer in _writers.items():
    if cid == exclude: continue
    writer.write(data) # same TCP conn
```

TCP **TCP — Java**

```java
for (var e : clients.entrySet()) {
  if (e.getKey() == excludeId) continue;
  e.getValue().out.write(data); // same
      socket
  e.getValue().out.flush();
}
```

QUIC **QUIC — Python**

```python
for proto in active_connections.values()
    :
    sid = proto._quic \
        .get_next_available_stream_id(
            is_unidirectional=True)
    proto._quic.send_stream_data(
        sid, data, end_stream=True)
    proto.transmit()
```

QUIC **QUIC — Java**

```java
QuicStream s = connection.createStream(
    false);
s.getOutputStream()
 .write((msg + "\n").getBytes());
s.getOutputStream().close(); // end-of-
    stream
```

TCP reuses one byte stream. QUIC opens a **new stream per broadcast** — no head-of-line blocking between messages.

Implementation Comparison — All Four Versions

	TCP Python	TCP Java	QUIC Python	QUIC Java
Library	asyncio	java.net	aioquic	kwik
Concurrency	Coroutines	Threads	Coroutines	Threads
Server listen	start_server	ServerSocket	serve()	ServerConnector
Per-conn object	coroutine	Runnable	Protocol subclass	AppProtoConnection
Receive data	readline()	readLine()	quic_event_received	acceptPeerInitiatedStream
Send data	writer.write	out.write	send_stream_data	createStream+write
TLS	None	None	Auto cert	keytool cert
Extra deps	None	None	aioquic	kwik (Maven)

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.
2. **QUIC $\neq$ "faster TCP".**
   The main benefit is *stream multiplexing* (no HoL blocking) and *0/1-RTT setup*, not raw throughput.

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.
2. **QUIC $\neq$ "faster TCP".**
   The main benefit is *stream multiplexing* (no HoL blocking) and *0/1-RTT setup*, not raw throughput.
3. **QUIC always uses TLS 1.3.**
   The certificate is part of the protocol — you cannot disable encryption.

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.
2. **QUIC $\neq$ "faster TCP".**
   The main benefit is *stream multiplexing* (no HoL blocking) and *0/1-RTT setup*, not raw throughput.
3. **QUIC always uses TLS 1.3.**
   The certificate is part of the protocol — you cannot disable encryption.
4. **Event-driven vs stream-driven.**
   TCP gives a continuous byte stream. QUIC delivers data as events. Libraries like kwik bridge this with `InputStream`.

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.
2. **QUIC $\neq$ "faster TCP".**
   The main benefit is *stream multiplexing* (no HoL blocking) and *0/1-RTT setup*, not raw throughput.
3. **QUIC always uses TLS 1.3.**
   The certificate is part of the protocol — you cannot disable encryption.
4. **Event-driven vs stream-driven.**
   TCP gives a continuous byte stream. QUIC delivers data as events. Libraries like kwik bridge this with `InputStream`.
5. **Coroutines (Python) vs Threads (Java).**
   Both handle many clients concurrently — different scheduling models.

Key Takeaways

1. **Same application logic, different transport API.**
   Broadcast chat works identically over TCP and QUIC — only the API changes.
2. **QUIC $\neq$ "faster TCP".**
   The main benefit is *stream multiplexing* (no HoL blocking) and *0/1-RTT setup*, not raw throughput.
3. **QUIC always uses TLS 1.3.**
   The certificate is part of the protocol — you cannot disable encryption.
4. **Event-driven vs stream-driven.**
   TCP gives a continuous byte stream. QUIC delivers data as events. Libraries like kwik bridge this with `InputStream`.
5. **Coroutines (Python) vs Threads (Java).**
   Both handle many clients concurrently — different scheduling models.
6. **QUIC is the future.**
   HTTP/3 (RFC 9114) is built on QUIC. Used by $\sim$25% of all web traffic today.

Try It Yourself

TCP — Python

```
python server_tcp.py
python client_tcp.py
```

TCP — Java

```
javac ServerTCP.java ClientTCP.java
java ServerTCP
java ClientTCP
```

QUIC — Python

```
pip install aioquic cryptography
python server.py
python client.py
```

QUIC — Java

```
cd quic_java && mvn package -q
java -cp target/quic-chat-*-jar-with-\
dependencies.jar ServerQUIC
java -cp target/quic-chat-*-jar-with-\
dependencies.jar ClientQUIC
```

github.com/ensarg/computer_networks_lab